Tarea 1 2026
Introducción a la Computación Paralela
Multiplicación de Matrices: Algoritmos Secuenciales y Paralelos

Cecilia Hernández y Álvaro Guzmán

Abril 2026

Fecha Inicio: Jueves 9 de Abril, 2026
Fecha Entrega: Lunes 27 de Abril, 2026
Grupos: de 3 estudiantes.

1 Descripción general

El objetivo general de esta tarea es implementar, analizar y comparar distintos algoritmos para la multiplicación de matrices densas cuadradas, tanto en sus variantes secuenciales como paralelas. En particular, se busca estudiar cómo influyen la localidad de memoria, el uso de submatrices o bloques, y la paralelización por tareas o por partición de datos en el rendimiento práctico sobre arquitecturas multicore.

Sean $A, B \in \mathbb{R}^{n \times n}$ y sea

$$C = A \times B,$$

donde

$$C[i, j] = \sum_{k=1}^{n} A[i, k]B[k, j].$$

En esta tarea se deben implementar y comparar las siguientes alternativas:

- **Secuenciales:**
  - multiplicación clásica;
  - multiplicación amigable a la caché usando submatrices o bloques;
  - algoritmo de Strassen.
- **Paralelas usando OpenMP:**
  - versión paralela de multiplicación basada en submatrices;
  - versión paralela de Strassen.
  - versión híbrida.

El trabajo debe incluir análisis teórico, estudio experimental y profiling de las implementaciones.

1.1 Objetivos

- Comprender diferencias entre algoritmos con igual complejidad asintótica pero distinto comportamiento en memoria.
- Comprender las diferencias entre algoritmos con distintas complejidades asintóticas y analizar el contraste entre su comportamiento teórico y su rendimiento en la práctica.
- Analizar el efecto de la localidad espacial y temporal en el desempeño.
- Implementar y evaluar paralelismo usando OpenMP.
- Relacionar resultados experimentales con análisis teórico.

2 Parte 1: Implementación de algoritmos secuenciales [2.0 pts]

Implemente las siguientes variantes secuenciales para matrices cuadradas densas:

2.1 Multiplicación clásica

Implemente la versión tradicional de triple ciclo anidado para calcular $C = A \times B$.

2.2 Multiplicación amigable a la caché usando submatrices

Implemente una versión por bloques (*tiling* o *blocking*), donde las matrices se procesen por submatrices cuadradas de tamaño $b \times b$ para mejorar la localidad de caché.

Sea b el tamaño de bloque. La idea general es recorrer las matrices por bloques:

$$C_{ij} \leftarrow C_{ij} + A_{ik}B_{kj},$$

donde cada A_{ik}, B_{kj}, C_{ij} es una submatriz de tamaño $b \times b$.

Se espera que el código permita variar el tamaño de bloque y estudiar su efecto en el rendimiento.

2.3 Algoritmo de Strassen

Implemente el algoritmo de Strassen para matrices cuadradas de dimensión potencia de 2. Si desea soportar tamaños arbitrarios, puede completar con ceros hasta la siguiente potencia de 2.

Recuerde que Strassen divide:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

y reduce el número de multiplicaciones recursivas de 8 a 7.

Además, debe definir un **caso base** a partir del cual se utilice un algoritmo secuencial tradicional o por bloques.

El algoritmo de Strassen se presenta a continuación:

Algorithm 1 Algoritmo de Strassen para multiplicación de matrices

Require: Matrices cuadradas $A, B \in \mathbb{R}^{n \times n}$, con n potencia de 2
Ensure: Matriz $C = A \cdot B$

function $\textsc{Strassen}(A, B, n)$
  if $n \leq n_0$ **then**
    return $\textsc{MultClasica}(A, B)$
  end if
  Dividir A en submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

  Dividir B en submatrices:

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

  Calcular los 7 productos de Strassen:

$$M_1 = \textsc{Strassen}(A_{11} + A_{22},\, B_{11} + B_{22},\, n/2)$$

$$M_2 = \textsc{Strassen}(A_{21} + A_{22},\, B_{11},\, n/2)$$

$$M_3 = \textsc{Strassen}(A_{11},\, B_{12} - B_{22},\, n/2)$$

$$M_4 = \textsc{Strassen}(A_{22},\, B_{21} - B_{11},\, n/2)$$

$$M_5 = \textsc{Strassen}(A_{11} + A_{12},\, B_{22},\, n/2)$$

$$M_6 = \textsc{Strassen}(A_{21} - A_{11},\, B_{11} + B_{12},\, n/2)$$

$$M_7 = \textsc{Strassen}(A_{12} - A_{22},\, B_{21} + B_{22},\, n/2)$$

  Combinar resultados:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

  Construir

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

  return C
end function

2.4 Requerimientos experimentales

Para las tres versiones secuenciales:

- Considere distintos tamaños de matrices, por ejemplo:

$$n \in \{256, 512, 1024, 2048, 4096\},$$

o un conjunto similar acorde a la memoria disponible.

- Ejecute múltiples repeticiones por experimento y reporte promedios.
- Verifique la corrección de las implementaciones comparando los resultados numéricamente.

2.5 Preguntas a responder

1. Compare la complejidad teórica de las tres variantes.
2. Explique por qué la versión por bloques puede ser más rápida que la clásica, aun cuando ambas realizan $\Theta(n^3)$ operaciones aritméticas.
3. Discuta cómo influye el tamaño de bloque b en el rendimiento.
4. Analice en qué rangos de n Strassen comienza o no a ser competitivo.

3 Parte 2: Implementación de algoritmos paralelos [2.0 pts]

Implemente las siguientes versiones paralelas usando OpenMP.

3.1 Multiplicación paralela basada en submatrices

Paralelice la multiplicación por bloques distribuyendo trabajo entre hebras. La estrategia puede basarse en paralelizar:

- los bloques de la matriz resultado;
- o bien los ciclos externos del algoritmo por bloques.

Debe asegurarse de que la paralelización no introduzca condiciones de carrera.

3.2 Strassen paralelo

Implemente una versión paralela del algoritmo de Strassen.

3.3 Versión paralela híbrida

Implemente una versión híbrida de la multiplicación de matrices que combine el algoritmo de Strassen con la multiplicación por bloques (cache-friendly).

3.4 Análisis teórico

Para ambas versiones paralelas, estudie:

- PRAM
- Work/Span;
- tiempo paralelo estimado usando el Teorema de Brent.

Explique además qué partes del algoritmo son naturalmente paralelizables y cuáles limitan la escalabilidad.

3.5 Evaluación experimental

Para cada implementación paralela:

- mida tiempos usando distintos números de hebras;
- por ejemplo, use
$$p \in \{1, 2, 4, 8\};$$
- calcule **speedup**
$$\text{Speedup}(p) = \frac{T_1}{T_p}$$
y **eficiencia**
$$\text{Eficiencia}(p) = \frac{\text{Speedup}(p)}{p}.$$

3.6 Preguntas a responder

1. Compare las estrategias de paralelización usadas en la versión por bloques y en Strassen.
2. Discuta cuál de las dos alternativas ofrece mejor escalabilidad práctica y por qué.
3. Explique cómo influye el umbral de corte en Strassen paralelo.
4. Relacione los resultados observados con el análisis de Work, Span y Brent.

4 Parte 3: Profiling y análisis de rendimiento [1.0 pts]

Realice profiling de sus implementaciones usando `perf` en Linux.

Debe incluir, al menos, mediciones de:

- ciclos (`cycles`),
- instrucciones (`instructions`),
- fallas de caché (`cache-misses`),
- referencias a caché (`cache-references`).

Puede usar comandos como:

```
perf stat ./programa
perf stat -e cycles,instructions,cache-references,cache-misses ./programa
```

Si lo desea, también puede incorporar eventos adicionales relevantes.

4.1 Preguntas a responder

1. Compare el comportamiento de caché entre la versión clásica y la versión por bloques.
2. Analice si una disminución en fallas de caché se traduce en una reducción del tiempo total.
3. Discuta cómo cambia el comportamiento al pasar de la versión secuencial a la paralela.

5 Informe y discusión final [1.0 pts]

El informe debe contener una discusión ordenada y clara de los resultados. Debe incluir:

- descripción de las implementaciones;
- decisiones de diseño;
- parámetros experimentales;
- tablas o gráficos con tiempos, speedup y eficiencia;
- resultados de profiling;
- interpretación crítica de los resultados.

5.1 Discusión

1. ¿Cuál fue la mejor alternativa secuencial en sus experimentos?
2. ¿Cuál fue la mejor alternativa paralela?
3. ¿Qué factores prácticos explican las diferencias con respecto a la complejidad teórica?
4. Si tuviera que recomendar una implementación para matrices grandes en un multicore, ¿cuál elegiría y por qué?

6 Requisitos de implementación y entrega

6.1 Implementación

- El lenguaje de implementación debe ser C o C++.
- Las versiones paralelas deben usar OpenMP.
- Debe indicarse claramente cómo compilar y ejecutar cada versión.
- Debe crear un repositorio con un readme, con sus implementaciones y sus scripts para reproducir sus experimentos.

6.2 Archivos a entregar

Solo su informe en pdf que incluya el link a su repositorio.

6.3 Formato del informe

El informe debe incluir:

- introducción breve;
- descripción de algoritmos;
- análisis teórico;
- metodología experimental;
- resultados;
- conclusiones.

7 Criterios de evaluación

Se evaluará:

- correctitud de las implementaciones;
- calidad del análisis teórico;
- calidad y claridad del estudio experimental;
- profundidad de la discusión de resultados;
- presentación general del informe.